SolarShare — Backend Schema

Version: 1.0 | **Database:** PostgreSQL (relational core) | **Companion to:** TRD v1.0

---

1. Entity Overview

```
Society ──< User ──< Meter ──< EnergyListing ──< Trade >── User
(buyer)
                                                    │
                                                    ├──< LedgerRecord
                                                    └──<
PaymentSettlement

User ──< Notification
```

Core entities:

1. `societies` — housing society / pilot zone
2. `users` — prosumers, buyers, and admins
3. `discom_verifications` — links a user to a verified DISCOM account (identity trust layer)
4. `meters` — net/smart meter records per user
5. `energy_listings` — surplus offered for sale
6. `trades` — matched buyer-seller transactions
7. `ledger_records` — tamper-evident hash log of every trade
8. `payment_settlements` — escrow/payment status per trade
9. `notifications` — WhatsApp/push notification log

2. Table Definitions

2.1 `societies`

Field	Type	Notes
`society_id`	UUID (PK)	
`name`	VARCHAR(150)	e.g., "Green Meadows RWA"
`discom_area`	VARCHAR(100)	e.g., "TPDDL", "PVVNL", "BRPL"

Field	Type	Notes
`pilot_zone`	BOOLEAN	true if within a DERC/UPERC-approved pilot area
`created_at`	TIMESTAMP	

2.2 `users`

Field	Type	Notes
`user_id`	UUID (PK)	
`society_id`	UUID (FK → societies)	
`full_name`	VARCHAR(150)	
`phone_number`	VARCHAR(15)	used for WhatsApp bot identity
`role`	ENUM('prosumer','buyer','admin')	a user can hold both prosumer + buyer flags via `is_prosumer` / `is_buyer` booleans if dual-role is needed
`is_prosumer`	BOOLEAN	
`is_buyer`	BOOLEAN	
`preferred_language`	VARCHAR(20)	for vernacular UI/WhatsApp bot
`created_at`	TIMESTAMP	

2.3 `discom_verifications`

Field	Type	Notes
`verification_id`	UUID (PK)	
`user_id`	UUID (FK → users)	
`discom_account_number`	VARCHAR(50)	

Field	Type	Notes
`verified_credential_hash`	VARCHAR(128)	mirrors IES's "verified digital identity" concept
`status`	ENUM('pending','verified','rejected')	
`verified_at`	TIMESTAMP NULL	

2.4 `meters`

Field	Type	Notes
`meter_id`	UUID (PK)	
`user_id`	UUID (FK → users)	
`meter_type`	ENUM('net_meter','smart_meter')	
`meter_serial_no`	VARCHAR(50)	as issued by DISCOM
`accuracy_class`	VARCHAR(10)	e.g., "1.0", "0.5S" per CEA Smart Meter Regulations 2023
`dlms_cosem_id`	VARCHAR(100)	device identifier on the DISCOM's AMISP/Head-End System
`is_bidirectional`	BOOLEAN	required for any prosumer meter
`last_reading_kwh_import`	DECIMAL(10,3)	
`last_reading_kwh_export`	DECIMAL(10,3)	
`last_synced_at`	TIMESTAMP	

2.5 `energy_listings`

Field	Type	Notes
`listing_id`	UUID (PK)	
`seller_id`	UUID (FK → users)	
`meter_id`	UUID (FK → meters)	
`kwh_available`	DECIMAL(8,3)	
`asking_price_per_kwh`	DECIMAL(6,2)	in ₹
`status`	ENUM('open','partially_matched','matched','expired')	
`listed_at`	TIMESTAMP	
`expires_at`	TIMESTAMP	listings tied to a generatio window (e.g., same-da

2.6 `trades`

Field	Type	Notes
`trade_id`	UUID (PK)	
`listing_id`	UUID (FK → energy_listings)	
`buyer_id`	UUID (FK → users)	
`seller_id`	UUID (FK → users)	
`kwh_traded`	DECIMAL(8,3)	
`agreed_price_per_kwh`	DECIMAL(6,2)	
`platform_fee_pct`	DECIMAL(4,2)	2–4% per Revenue Model
`status`	ENUM('pending','confirmed','settled','disputed')	

Field	Type	Notes
`created_at`	TIMESTAMP	
`confirmed_at`	TIMESTAMP NULL	

2.7 `ledger_records`

Field	Type	Notes
`ledger_id`	UUID (PK)	
`trade_id`	UUID (FK → trades)	
`record_hash`	VARCHAR(128)	SHA-256 of trade payload
`previous_hash`	VARCHAR(128) NULL	chains to prior record — tamper evidence
`recorded_at`	TIMESTAMP	

2.8 `payment_settlements`

Field	Type	Notes
`settlement_id`	UUID (PK)	
`trade_id`	UUID (FK → trades)	
`escrow_reference`	VARCHAR(100)	payment gateway transaction reference
`amount`	DECIMAL(10,2)	
`status`	ENUM('held','released','refunded')	
`discom_bill_cycle_ref`	VARCHAR(50) NULL	links to the billing cycle where the adjustment appears
`settled_at`	TIMESTAMP NULL	

2.9 `notifications`

Field	Type	Notes
`notification_id`	UUID (PK)	
`user_id`	UUID (FK → users)	
`channel`	ENUM('whatsapp','push','sms')	
`message`	TEXT	
`sent_at`	TIMESTAMP	

3. Sample SQL DDL (Core Tables)

```
CREATE TABLE societies (
    society_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(150) NOT NULL,
    discom_area VARCHAR(100),
    pilot_zone BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID REFERENCES societies(society_id),
    full_name VARCHAR(150) NOT NULL,
    phone_number VARCHAR(15) UNIQUE NOT NULL,
    is_prosumer BOOLEAN DEFAULT FALSE,
    is_buyer BOOLEAN DEFAULT FALSE,
    preferred_language VARCHAR(20) DEFAULT 'en',
    created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE meters (
    meter_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id),
    meter_type VARCHAR(20) CHECK (meter_type IN
('net_meter','smart_meter')),
    meter_serial_no VARCHAR(50),
    accuracy_class VARCHAR(10),
    is_bidirectional BOOLEAN DEFAULT FALSE,
    last_reading_kwh_import DECIMAL(10,3) DEFAULT 0,
    last_reading_kwh_export DECIMAL(10,3) DEFAULT 0,
    last_synced_at TIMESTAMP
);

CREATE TABLE energy_listings (
    listing_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    seller_id UUID REFERENCES users(user_id),
```

```
    meter_id UUID REFERENCES meters(meter_id),
    kwh_available DECIMAL(8,3) NOT NULL,
    asking_price_per_kwh DECIMAL(6,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'open',
    listed_at TIMESTAMP DEFAULT now(),
    expires_at TIMESTAMP
);

CREATE TABLE trades (
    trade_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    listing_id UUID REFERENCES energy_listings(listing_id),
    buyer_id UUID REFERENCES users(user_id),
    seller_id UUID REFERENCES users(user_id),
    kwh_traded DECIMAL(8,3) NOT NULL,
    agreed_price_per_kwh DECIMAL(6,2) NOT NULL,
    platform_fee_pct DECIMAL(4,2) DEFAULT 3.00,
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT now(),
    confirmed_at TIMESTAMP
);

CREATE TABLE ledger_records (
    ledger_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trade_id UUID REFERENCES trades(trade_id),
    record_hash VARCHAR(128) NOT NULL,
    previous_hash VARCHAR(128),
    recorded_at TIMESTAMP DEFAULT now()
);

CREATE TABLE payment_settlements (
    settlement_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trade_id UUID REFERENCES trades(trade_id),
    escrow_reference VARCHAR(100),
    amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'held',
    discom_bill_cycle_ref VARCHAR(50),
    settled_at TIMESTAMP
);
```

4. Notes on MVP vs. Pilot Schema

- **Hackathon MVP:** the above schema can run entirely on Firebase/Supabase free tier with `meters.last_reading_*` populated by a simulated data generator instead of a real DISCOM feed.
- **Pilot stage:** `meters` table gains a foreign sync job against the DISCOM's AMISP/Head-End System (DLMS/COSEM); `payment_settlements` connects to a real escrow-based payment aggregator instead of a mock wallet.

- **Indexing priorities:** `energy_listings(status, expires_at)` and `trades(status, created_at)` should be indexed first — these are the highest-frequency queries (active marketplace view, pending settlement checks).